C PROGRAMMING PROJECT PRESENTATION

Project Title: Billing and Receipt
Course: Mini Project
Student Name & Roll No: Rohith(25bcs141),Roshini kumari(25bcs142), Sachin (25bcs143), Sanjay (25bcs144),Santhanalakshmi (25bcs145)
Department: Computer Science
Mentor / Guide: Ashwanandini

INTRODUCTION

- Overview of the project and its purpose
- Capturing customer and transaction details.Generating invoices list all goods/services provided, including prices, taxes, and payment terms.Sending invoices to customers through digital or physical channels.Recording payments and issuing receipts as proof of settlement.
- Problem background and motivation
- Billing and receipts are fundamental aspects of financial operations for any business, serving to request payment and confirm completed transactions, respectively.
- Business Growth: Scalable billing infrastructure and automation allow businesses to handle growth, manage complex subscriptions, and expand globally without being hampered by back-office failures.
- Significance of the project in solving real-world issue
- Financial Viability and Cash Flow
- Client Trust and Transparency

OBJECTIVE

- Main goals of the project:
- Automating the calculation of total charges based on items or services consumed, such as quantity and unit price or usage minutes.
- Managing customer details such as name, phone number, address, and billing history.
- Generating clear, formatted receipts or bills showing itemized costs, total amount due, payment status, and other transaction details.

PROBLEM STATEMENT

- Describe the current issue or gap in existing systems
- Manual data entry and tracking that reduces efficiency.Errors like incorrect item descriptions or prices causing payment delays and disputes.
- Identify limitations or inefficiencies
- Late payments due to unclear due dates or unmailed invoicesInvoice errors like duplicate charges, wrong customer details, or incorrect taxesManual entry errors causing discrepancies and delays

SYSTEM REQUIREMENTS

- Hardware Requirements:
- • Processor: Intel i3 or higher
- • RAM: 4 GB minimum
- • Storage: 500 MB free space

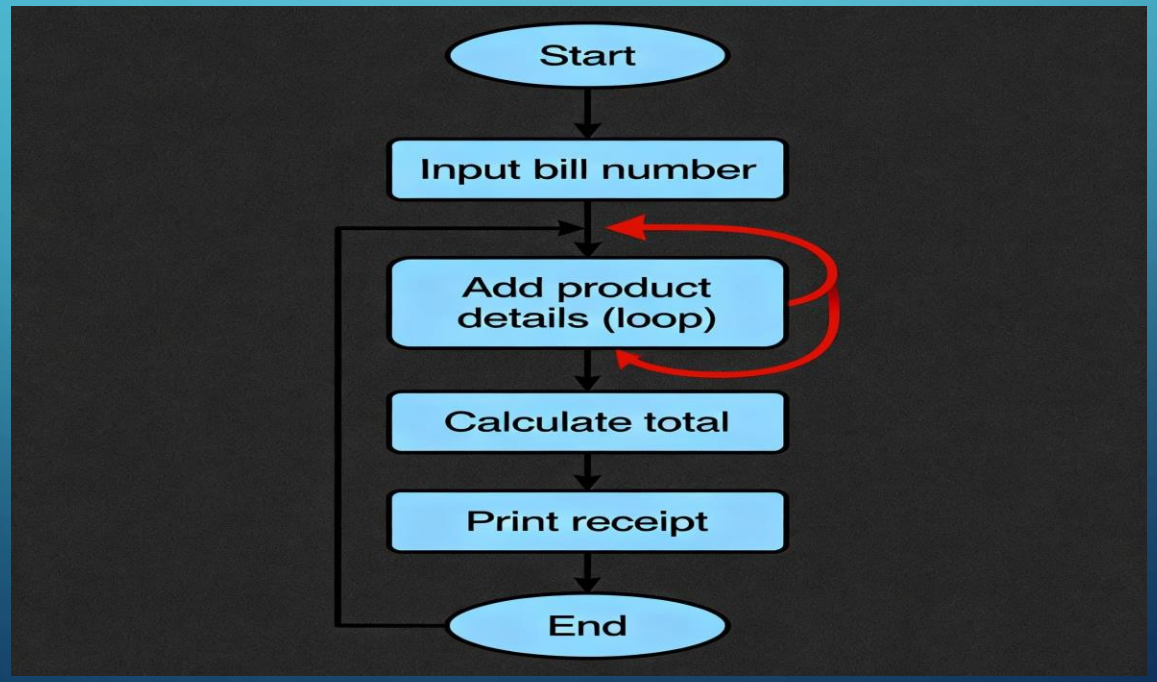
- Software Requirements:
- • OS: Windows / Linux
- • Compiler: GCC / Turbo C / Code::Blocks / Dev C++

SYSTEM DESIGN

- Structure and workflow of the system
- Initialize a bill with a unique bill number
- Add products with details (id, name, price, quantity) to the bill
- Calculate the total amount based on product prices and quantities.
- Print the receipt showing all products, their amounts, and total.

FLOWCHART DESCRIPTION AND BLOCK DIAGRAM

- Flowchart DescriptionStart nodeInput bill numberLoop: Input product details (id, name, price, quantity)Add product to listCheck if more products to add: Yes — continue loop; No — next stepCalculate totalPrint receiptEnd node



ALGORITHM OUTLINE

- Start
- Initialize bill structure with bill number and product count zero
- Repeat input for each product.
- Take product id, name, price, and quantity
- Add product to bill
- Calculate the total amount by summing product price * quantity for all products
- Print the receipt with product details and total amount
- End

SOURCE CODE

```
#include <stdio.h>
#include <string.h>

struct Product {
    int id;
    char name[50];
    float price;
    int quantity;
};

struct Bill {
    int billNumber;
    struct Product products[100];
    int productCount;
    float totalAmount;
};
```

```
Void addProduct(struct Bill *bill, int id, char *name, float price, int quantity) {
    struct Product p;
    p.id = id;
    strcpy(p.name, name);
    p.price = price;
    p.quantity = quantity;
    bill->products[bill->productCount++] = p;
}

void calculateTotal(struct Bill *bill) {
    bill->totalAmount = 0;
    for (int i = 0; i < bill->productCount; i++) {
        bill->totalAmount += bill->products[i].price * bill->products[i].quantity;
    }
}
```

```
Void printReceipt(struct Bill *bill) {
    printf("
Receipt – Bill No: %d
", bill->billNumber);
    printf("%-10s %-20s %-10s %-10s %-10s
", "ID", "Product", "Price", "Quantity", "Amount");
    for (int i = 0; i < bill->productCount; i++) {
        struct Product p = bill->products[i];
        printf("%-10d %-20s %-10.2f %-10d %-10.2f
", p.id, p.name, p.price, p.quantity, p.price * p.quantity);
    }
    printf("
Total Amount: %.2f
", bill->totalAmount);
}
```

```
Int main() {
    struct Bill bill;
    bill.billNumber = 1;  // example bill number
    bill.productCount = 0;

    // Example adding products
    addProduct(&bill, 101, “Milk”, 30.5, 2);
    addProduct(&bill, 102, “Bread”, 20.0, 1);
    addProduct(&bill, 103, “Eggs”, 5.0, 12);

    calculateTotal(&bill);
    printReceipt(&bill);

    return 0;
}
```

MODULE DESCRIPTION

- Input ModuleTakes user input for bill number and product details (id, name, price, quantity).The function addProduct encapsulates adding individual products to the bill.Input can be extended to actively prompt the user in a loop to add multiple products.
- Processing ModulePerforms necessary calculations and logic.The calculateTotal function computes the total bill amount by summing the price multiplied by quantity for each product.Can be extended for discounts, taxes, or other business logic as needed.
- Output ModuleDisplays results to users in a readable format.The printReceipt function shows a detailed receipt with product-wise amounts and total bill.It formats output in columns for clarity.
- File Handling Module (Suggested Addition)This module would manage saving bills and products to files and retrieving them later.Functions to open file in write mode and save the Bill structure, and read from file to load past bills can be implemented.This is useful for persistence and tracking past transactions.

OUTPUT SCREEN

- Receipt – Bill No: 1
- ID Product Price Quantity Amount
- 101 Milk 30.50 2 61.00
- 102 Bread 20.00 1 20.00
- 103 Eggs 5.00 12 60.00
-
- Total Amount: 141.00

SUMMARY OF OUTPUT

- Bill number header for identification
- Each product's ID, name, unit price, quantity purchased, and the total price per item (price × quantity).
- A final line displaying the sum total of all products.

OBSERVATION

- The output is user-friendly and well-formatted in tabular form, making it easy to read and understand the billing details. The calculation of total amount correctly adds up the individual product amounts.

CHALLENGES FACED

- Ensuring proper alignment of text and numbers for readability required careful use of formatted print specifiers.Managing a dynamic list of products within a fixed-size array limited to 100 products; more robust data structures like linked lists would be better for real-world scenarios

Adding modularity for file handling to save and retrieve bills, which was not included in the base program but is important for persistent billing systems

User interaction is minimal (hard-coded product additions); expanding to interactive input validation and error handling adds complexity

Handling memory and string operations carefully to avoid buffer overflow or invalid memory access.

CONCLUSION

- The C program for the billing and receipt system effectively demonstrates the use of structures to organize related data (products and bills), modular programming with functions for adding products, calculating totals, and printing receipts. The output is well-formatted and clear for users, making it practical for basic billing purposes. The program serves as a fundamental example of managing data, performing calculations, and presenting results in a readable format.

FUTURE ENHANCEMENT

- Interactive User Input: Instead of hardcoded products, implement an input module to allow users to enter product details dynamically.
- File Handling: Add modules to save bills to files and retrieve past bills for persistent data storage.
- Error Handling: Incorporate validation checks on inputs to handle invalid data gracefully.
- Discounts and Taxes: Implement calculations for discounts, taxes, or other financial rules.
- Scalability: Use dynamic data structures like linked lists or arrays with dynamic memory allocation for an unlimited number of products.
- Graphical Interface: Develop a GUI for better user interaction using libraries like GTK or ncurses.
- Reports and Analytics: Add features to generate summaries and reports for sales analytics.

REFERENCES

- • Let Us C – Yashavant Kanetkar
- • C Programming – E. Balagurusamy
- • Online resources and documentation

- Acknowledgement:
- • Express gratitude to mentor, department, and team members

QUESTION

- How can file handling be integrated into the billing system, and what are best practices for storing bill data?
- What data structures are most suitable when the number of products is unknown or very large?

THANK YOU